# Interrupts, Timers & PWM

Day 2 → 

Day 2 — Embedded Systems 7-Day Study Series

## 7-Day Course Roadmap

| Day | Topic |
|---|---|
| Day 1 ✓ | Introduction — Architecture, GPIO, Memory, Protocols |
| Day 2 → | Interrupts, Timers & PWM |
| Day 3 | UART, SPI, I2C — Deep Dive |
| Day 4 | ADC/DAC & Sensor Interfacing |
| Day 5 | RTOS Fundamentals (FreeRTOS) |
| Day 6 | Power Management & Low-Power Design |
| Day 7 | Bootloaders, OTA & System Design Project |

## ■ 1. Interrupts

**What is an Interrupt?** An interrupt is a hardware or software signal that temporarily suspends the CPU's current task and transfers control to an **Interrupt Service Routine (ISR)**. After the ISR completes, the CPU resumes the previous task from exactly where it left off.

| Type | Source | Priority | Latency |
|---|---|---|---|
| Hardware (External) | GPIO pin, EXTI line | Configurable | Low |
| Timer Interrupt | Timer overflow/compare | Configurable | Low |
| DMA Interrupt | Transfer complete | High | Very Low |
| Software Interrupt | SVC, PENDSV | Fixed | Medium |
| Fault Exception | HardFault, BusFault | Highest | Immediate |

**ISR Best Practices:**

- Keep ISRs **short and fast** — never block or use delays inside an ISR.
- Use **volatile** keyword for variables shared between ISR and main code.
- Disable global interrupts (**__disable_irq()**) only when strictly necessary.
- Use **flags** in ISR; process the actual work in the main loop.
- Clear the interrupt **pending bit** before exiting the ISR.

**NVIC Configuration (STM32 example):**

```
HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0); // preempt=0, sub=0
HAL_NVIC_EnableIRQ(EXTI0_IRQn);
void EXTI0_IRQHandler(void) {
```

```
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
}
```

> **KEY CONCEPT**
>
> Always set interrupt priorities carefully. On ARM Cortex-M, lower numeric value = higher priority. Peripherals sharing data should use the same priority group.

## ■ 2. Timers

Timers are hardware counters driven by the system clock (or a prescaled version). They are fundamental to **time measurement**, **event counting**, **PWM generation**, and **input capture**.

| Timer Mode | Use Case | Key Register |
|---|---|---|
| Up-counting | Delay / overflow interrupt | ARR (Auto-Reload) |
| Down-counting | Timeout detection | ARR, CNT |
| Centre-aligned | Symmetric PWM (BLDC motors) | ARR, CCR |
| Input Capture | Measure pulse width / frequency | CCR, CCMR |
| Output Compare | Generate precise waveforms | CCR, CCMR |
| Encoder Mode | Read quadrature encoder | SMCR |

**Timer Frequency Formula:**

```
Update Frequency = Timer_Clock / [(PSC + 1) x (ARR + 1)]

Example: 72 MHz clock, PSC=71, ARR=999 → 1000 Hz (1 ms tick)
```

> **NOTE**
>
> SysTick is a 24-bit dedicated system timer on Cortex-M. HAL uses it for HAL_Delay() and HAL_GetTick(). Avoid modifying SysTick when using HAL.

## ■ 3. PWM — Pulse Width Modulation

PWM encodes information in the **duty cycle** of a digital signal. By varying the ON-time vs OFF-time ratio, we can control average power delivered to a load — e.g. motor speed, LED brightness, servo position.

| Parameter | Formula | Typical Range |
|---|---|---|
| Period (T) | T = 1 / f | 1 µs – 20 ms |
| Duty Cycle (%) | (t_on / T) × 100 | 0 % – 100 % |
| CCR Value | CCR = (Duty/100) × (ARR+1) | 0 – ARR |
| Resolution | Bits = log2(ARR + 1) | 8 – 16 bit |

**PWM Applications:**

- **LED Dimming** — 1–10 kHz PWM; human eye can't detect flicker above ~60 Hz.
- **DC Motor Control** — 10–50 kHz; use H-bridge driver (L298N, DRV8833).
- **Servo Motor** — 50 Hz, 1–2 ms pulse width → 0°–180° rotation.
- **Audio Generation** — High-frequency PWM + RC filter = DAC substitute.
- **SMPS / Buck Converter** — 100 kHz–MHz switching for power regulation.

**HAL PWM Snippet:**

```
// Start PWM on TIM3 Channel 1
HAL_TIM_PWM_Start(&htim3;, TIM_CHANNEL_1);
// Set 50% duty cycle (ARR = 999)
TIM3->CCR1 = 500;
// Change to 75% duty cycle
TIM3->CCR1 = 750;
```

**SERVO TIP**

For servo control: 50 Hz period (ARR = 19999 at 1 MHz timer), CCR = 1000 for 0 deg, CCR = 1500 for 90 deg, CCR = 2000 for 180 deg.

## ■ 4. Practice Exercises

| # | Exercise | Difficulty | Est. Time |
|---|---|---|---|
| 1 | Configure EXTI on PA0 — toggle LED on button press (interrupt-driven, no p■■ng) | | 30 min |
| 2 | Use TIM2 to generate a 1-second tick; count seconds on UART terminal | ■■ | 45 min |
| 3 | PWM LED breathing effect — ramp duty 0→100→0 % using timer interrupt | ■■■ | 60 min |
| 4 | Input Capture: measure the period of an external square wave signal | ■■■ | 60 min |
| 5 | Servo sweep: move from 0° to 180° and back using 50 Hz PWM on TIM3 | ■■■ | 45 min |

## 5. Quick-Reference Cheat Sheet

| Concept | Key Point |
|---|---|
| `IRQ Priority (Cortex-M)` | Lower number = higher priority; 0 is highest |
| `Volatile keyword` | Required for ISR-shared variables to prevent optimisation |
| `Timer Clock Source` | APB1 or APB2 bus clock (check clock tree in CubeMX) |
| `ARR register` | Sets the timer period (overflow value) |
| `PSC register` | Prescaler — divides input clock before counter |
| `CCR register` | Capture/Compare — sets PWM duty or captures timestamp |
| `DMA vs Interrupt` | DMA offloads CPU; use for high-speed data (ADC, UART Rx) |
| `HAL_Delay()` | Blocking delay using SysTick; avoid inside ISR |
| `__HAL_TIM_SET_COMPARE()` | Macro to change CCR value at runtime for PWM |

## Day 2 Summary & Next Steps

| ■ Covered Today | ➡ Coming Up — Day 3 |
|---|---|
| • Interrupt types and NVIC priority<br>• Writing safe ISRs<br>• Timer modes: up/down/centre-aligned<br>• Input Capture & Output Compare<br>• PWM duty cycle & frequency maths<br>• Servo & motor PWM control | • UART framing: start/stop/parity bits<br>• SPI: CPOL/CPHA modes, NSS handling<br>• I2C: 7/10-bit addressing, ACK/NACK<br>• Multi-master bus arbitration<br>• HAL vs LL driver deep dive<br>• DMA-driven UART receive |

*Study tip: Complete exercises 1–3 today and exercises 4–5 as stretch goals. Review the STM32 Reference Manual sections on TIM and EXTI for deeper understanding.*